Experiment – 3
Geometric Transformations on Image

AIM: To perform geometric transformations such as scaling (resizing) and rotation of a digital image.

Theory

1. Image Scaling

Scaling is the process of enlarging or shrinking an image by a scaling factor along the horizontal and vertical directions. It resizes the image while preserving its overall structure. Common interpolation methods include nearest-neighbor (fastest but can produce aliasing), bilinear (better quality, moderate speed), and bicubic (higher quality, slower computation).

2. Image Rotation

Rotation involves turning an image by a specified angle around a fixed point, usually the image center. It is performed using an affine transformation matrix that maps the original pixel coordinates to new locations. Rotation is useful for alignment and orientation correction.

3. Image Resizing

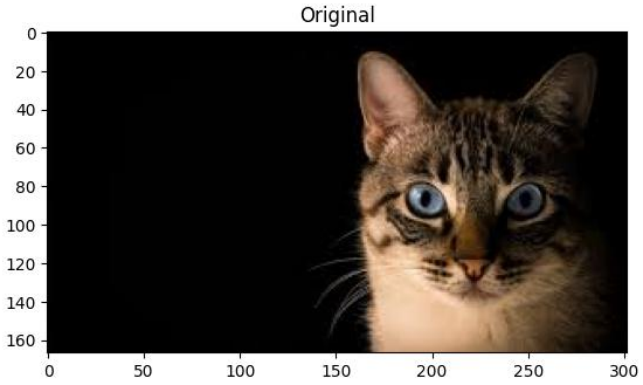
Resizing is the process of changing the image dimensions to a fixed size. Unlike scaling by a factor, resizing directly specifies the output width and height. Appropriate interpolation methods are used to preserve image quality during resizing.

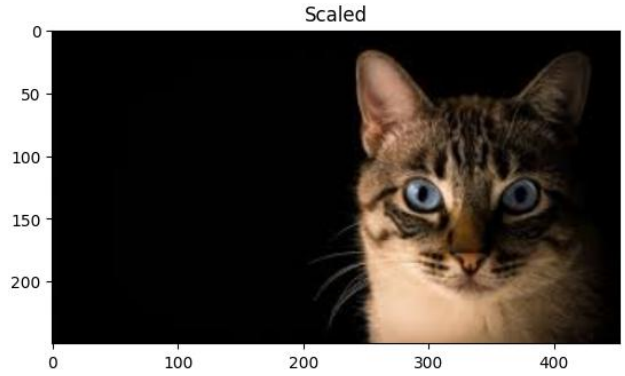
Code

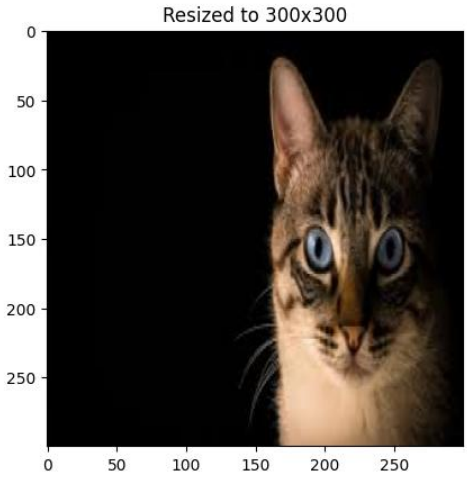
```
import cv2
import numpy as np
from matplotlib import pyplot as plt
%matplotlib inline

# Load image
img = cv2.imread("./content/cat.jpg")
if img is None:
    raise FileNotFoundError("cat.jpg not found")

# Display original image
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title("Original")
plt.show()
# Scaling (uniform)
scale_factor = 1.5
scaled = cv2.resize(img, None, fx=scale_factor, fy=scale_factor,
interpolation=cv2.INTER_LINEAR)
scaled_rgb = cv2.cvtColor(scaled, cv2.COLOR_BGR2RGB)
plt.imshow(scaled_rgb)
plt.title("Scaled")
plt.show()
```

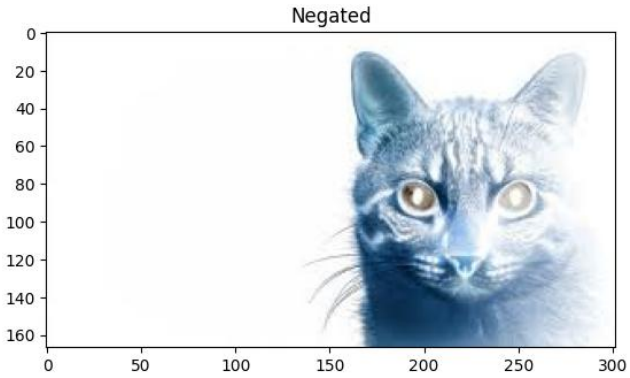
```
# Rotation (around center)
(h, w) = img.shape[:2]
center = (w // 2, h // 2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(img, M, (w, h))
rotated_rgb = cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB)
plt.imshow(rotated_rgb)
plt.title("Rotated")
plt.axis("off")
plt.show()

# Resizing to fixed size
resized = cv2.resize(img, (300, 300), interpolation=cv2.INTER_AREA)
resized_rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)
plt.imshow(resized_rgb)
plt.title("Resized to 300x300")
plt.show()

# Negating (invert colors)
negated = cv2.bitwise_not(img)
negated_rgb = cv2.cvtColor(negated, cv2.COLOR_BGR2RGB)
plt.imshow(negated_rgb)
plt.title("Negated")
plt.show()
```

Output









Results and Discussion

The experiment was successfully implemented to perform scaling, rotation, and resizing of an image. During scaling, the image was enlarged using nearest neighbor, bilinear, and bicubic interpolation methods. Nearest neighbor scaling produced a pixelated effect, while bilinear and bicubic interpolation resulted in smoother and visually improved images. The rotation operation rotated the image by 45 degrees about its center using an affine transformation. The rotated image maintained its spatial structure, confirming the correctness of the rotation matrix. Finally, the image was resized to a fixed dimension of 300 × 300 pixels using area-based interpolation, which preserved image quality during downscaling. Overall, the results validate the theoretical concepts of geometric image transformations and demonstrate the effect of different interpolation techniques on image quality.